PLEASE INSERT COIN

An Interactive 2D and 3D Vending Machine Mini-Game Using C++, OpenGL, and FreeGLUT



Submitted in Partial Fulfillment of the Requirements for

CS0045: Introduction to Computer Graphics and Visual Computing

Submitted by:

Earl John Viray

Section:

TN35

Submitted to:

Dr. Hadji Tejuco

Date of Submission:

October 12, 2026

Introduction

An Interactive 2D and 3D Vending Machine Mini-Game

The title *PLEASE INSERT COIN* comes from the familiar instruction displayed on vending machines and other coin-operated devices. I chose this title because it establishes the game's ordinary premise while also serving as an important part of its interface and presentation.

At first, the title suggests a simple vending-machine simulation. However, the gameplay introduces an unexpected change in perspective when an ordinary snack purchase becomes a three-dimensional recovery operation. This contrast between the familiar and the unusual is central to the project's identity.

The title also serves a narrative purpose. It appears at the beginning of the game and returns after the final sequence, creating a sense that the vending machine has resumed its normal operation despite the strange events that occurred.

Project Description

PLEASE INSERT COIN is an interactive mini-game developed using C++, OpenGL, and FreeGLUT. The application combines two-dimensional and three-dimensional computer graphics within a single gameplay experience, demonstrating how geometric primitives, vertex arrays, user-input callbacks, and state-based programming can be integrated into an interactive graphical system.

The game takes place on a quiet nighttime street, where customers approach an ordinary vending machine to purchase snacks. What initially appears to be a straightforward transaction takes an unexpected turn when the selected item becomes stuck inside the machine. The player is then transported into a three-dimensional maintenance environment and must operate a mechanical claw to retrieve the item and deliver it to the dispensing chute.

The application features three progressively challenging stages, each introducing a different customer, snack, and obstacle arrangement. Its visual presentation contrasts a warm, retro-inspired two-dimensional exterior with a surreal, low-poly three-dimensional interior.

The game also incorporates subtle environmental storytelling. As the stages progress, unusual customer dialogue and minor disturbances in the vending machine suggest that something is not entirely ordinary. These details culminate in a final visual sequence that leaves the underlying mystery open to interpretation.

Beyond its entertainment value, the project demonstrates the practical application of foundational computer graphics concepts, including geometric construction, coordinate systems, rendering, keyboard and mouse interaction, animation, collision detection, and vertex-array-based drawing.

Problem/Purpose

The primary purpose of this project is to demonstrate my understanding of the fundamental concepts introduced in CS0045: Introduction to Computer Graphics and Visual Computing by applying them to a complete interactive graphical application.

A major challenge in introductory computer graphics is moving beyond isolated demonstrations of shapes, colors, and drawing commands. Although these examples are essential for understanding how OpenGL works, they do not always demonstrate how different programming and graphics concepts interact within a functioning application.

I wanted to address this challenge by developing a small game in which the concepts covered throughout the course serve practical purposes.

Rather than drawing geometric objects solely for demonstration, I use them to construct a vending machine, customer characters, snack packages, mechanical equipment, and interactive environments. Keyboard and mouse callbacks allow the player to interact with these objects, while conditions and application states determine how the game responds to different actions.

The project also explores the relationship between two-dimensional and three-dimensional rendering. By transitioning between a 2D purchasing scene and a 3D rescue environment, I can demonstrate how different coordinate systems, projections, and rendering techniques can be used within the same application.

My objective is not to develop a commercially sophisticated game, but to create a cohesive interactive experience that meaningfully applies the programming and graphical techniques introduced in the course.

Target User

The intended users of *PLEASE INSERT COIN* are casual players, particularly those who enjoy short interactive games with unconventional mechanics, retro-inspired graphics, and subtle environmental storytelling.

The game is designed to be approachable even for players who have limited experience with computer games. Its initial interaction involves familiar actions, such as inserting a coin and selecting an item from a vending machine. The more involved rescue sequence introduces straightforward keyboard controls for moving and operating a mechanical claw.

The application is also intended for students and instructors studying introductory computer graphics. It demonstrates how foundational OpenGL techniques can be combined to create a functioning interactive system rather than a collection of unrelated graphical exercises.

By maintaining simple controls and a relatively short gameplay structure, I aim to make the project accessible to both casual players and individuals evaluating its technical implementation.

Brainstorming Summary

During the planning stage, I wanted to develop a project that would satisfy the technical requirements of CS0045 while also offering a recognizable and engaging gameplay experience.

I considered the importance of choosing a concept that could be constructed using OpenGL geometric primitives without depending on complex models, external graphics assets, or a full game engine.

My main consideration was how to make the application feel like a small interactive system rather than a conventional programming exercise. I wanted the objects, controls, and graphical features to contribute to an identifiable gameplay objective.

The vending-machine concept provided a practical foundation because it naturally supports several forms of interaction. A player can insert a coin, select an item, observe the machine's response, and receive feedback based on the selected action.

However, I felt that a straightforward vending-machine simulation would not make full use of the creative possibilities offered by computer graphics. I therefore developed the idea of having the machine malfunction, requiring the player to recover the selected item manually.

This introduced the opportunity to combine a conventional two-dimensional interface with a three-dimensional environment.

As the concept developed, I also explored how the game's visual presentation could support its narrative. I settled on a retro-inspired nighttime street for the ordinary purchasing sequence and a surreal, low-poly industrial interior for the recovery sequence.

The final direction combines familiar interactions, a sudden change in gameplay perspective, progressively challenging recovery stages, and a subtle mystery involving the customers and the vending machine.

Initial Idea

My initial concept was a small interactive game centered around an unreliable vending machine.

The player would interact with the machine through a simple purchasing process. A customer would request a particular snack, and the player would insert a coin and select the corresponding item.

Instead of successfully dispensing the product, the vending machine would malfunction and leave the selected snack stuck inside.

To resolve the problem, the player would gain control of a mechanical claw inside the machine. The objective would be to retrieve the stuck item and guide it toward the dispensing chute.

This concept was intended to introduce a clear gameplay objective while allowing me to demonstrate different OpenGL features. The vending machine and its surrounding environment could be constructed using two-dimensional geometric primitives, while the claw and snack-recovery system could use three-dimensional geometry.

The initial idea emphasized the core interaction between purchasing and recovering an item. Additional elements, such as the three-stage structure, detailed visual identity, contextual interfaces, and environmental storytelling, were developed during subsequent planning.

Final Idea

The final concept of *PLEASE INSERT COIN* expands the original vending-machine rescue idea into a three-stage interactive mini-game with a consistent visual identity, progressively challenging mechanics, and an underlying narrative mystery.

The game begins on a quiet nighttime street rendered in a retro arcade-inspired two-dimensional style. The vending machine is the primary visual element, illuminated by warm lighting that contrasts with the darker surrounding buildings and pavement.

Each stage introduces a customer who requests a specific snack. The player must insert a coin and select the correct item using the mouse. When the snack becomes stuck, the scene transitions unexpectedly into a three-dimensional representation of the machine's interior.

The interior is designed with stylized low-poly geometry and an exaggerated industrial appearance. The player controls a mechanical claw using the keyboard, positioning it above the requested snack, lowering it to attempt a grab, and transporting the item toward the dispensing chute.

The three stages introduce different recovery challenges:

Stage 1: A regular night. The player retrieves a bag of chips in a relatively open environment. This stage introduces the basic claw movement and recovery mechanics.

Stage 2: Precision Nutrition. The player retrieves a chocolate bar surrounded by additional snack packages. The arrangement requires more careful positioning and movement.

Stage 3: IT is thirsty. The player retrieves a juice carton while navigating around a moving restocking arm, introducing an additional obstacle to the recovery process.

The game uses limited attempts rather than a countdown timer. This allows the player to focus on positioning and understanding the controls without unnecessary time pressure.

The final design also incorporates a subtle narrative. The customers appear ordinary, but their dialogue and the vending machine's occasional electrical disturbances gradually introduce inconsistencies. These clues are intentionally ambiguous and do not affect the gameplay mechanics.

After the final customer receives their snack and leaves, their reflection remains visible in the vending machine's glass. It turns toward the player before disappearing during a brief flicker of light. The game then returns to its ordinary title screen without explaining the event.

Through this combination of interactive gameplay, geometric rendering, visual contrast, and environmental storytelling, I aim to create an application that demonstrates the course concepts while maintaining a distinct creative identity.

Sketch/Initial Design

The initial design of *PLEASE INSERT COIN* consists of two main graphical environments: a two-dimensional vending-machine scene and a three-dimensional snack-recovery environment.

Two-Dimensional Environment

The first environment uses a side-view composition inspired by retro arcade games. The vending machine occupies the central portion of the screen, while the surrounding nighttime street provides atmosphere and visual context.

The machine includes a cabinet, glass display, snack shelves, selection buttons, coin slot, and dispensing chute. A customer stands beside the machine, and a dialogue area presents the customer's request and relevant instructions.

The arrangement emphasizes the vending machine as the main interactive object while keeping the purchasing controls visible and accessible.

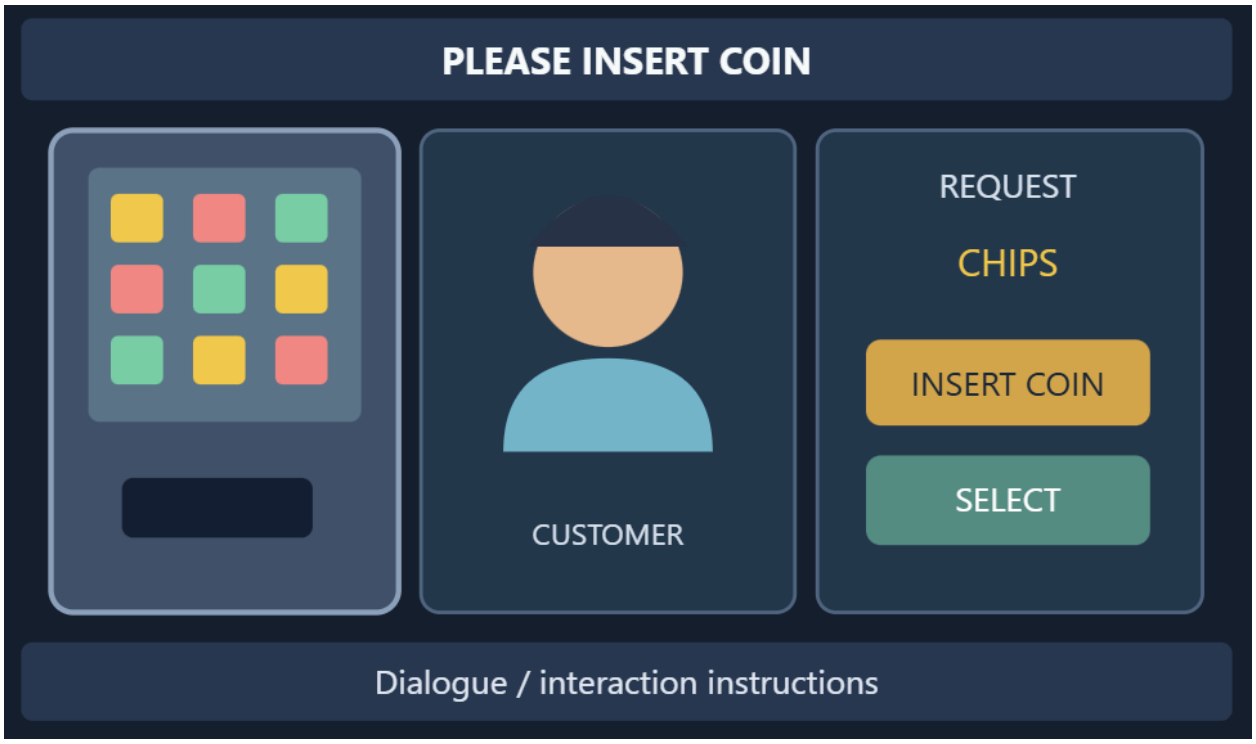


Figure 1. Initial design of the nighttime street and vending-machine purchasing interface.

Three-Dimensional Environment

The second environment represents the interior of the vending machine as an unusually large industrial compartment.

It uses a fixed, elevated oblique camera to provide a clear view of the mechanical claw, snack packages, surrounding obstacles, and dispensing chute.

The claw moves along the horizontal and depth axes, while its vertical movement is used during the grabbing process. A projected marker beneath the claw helps the player estimate its position relative to the target.

The environment uses angular geometric objects, simplified shading, and contrasting colors to establish a low-poly visual style.

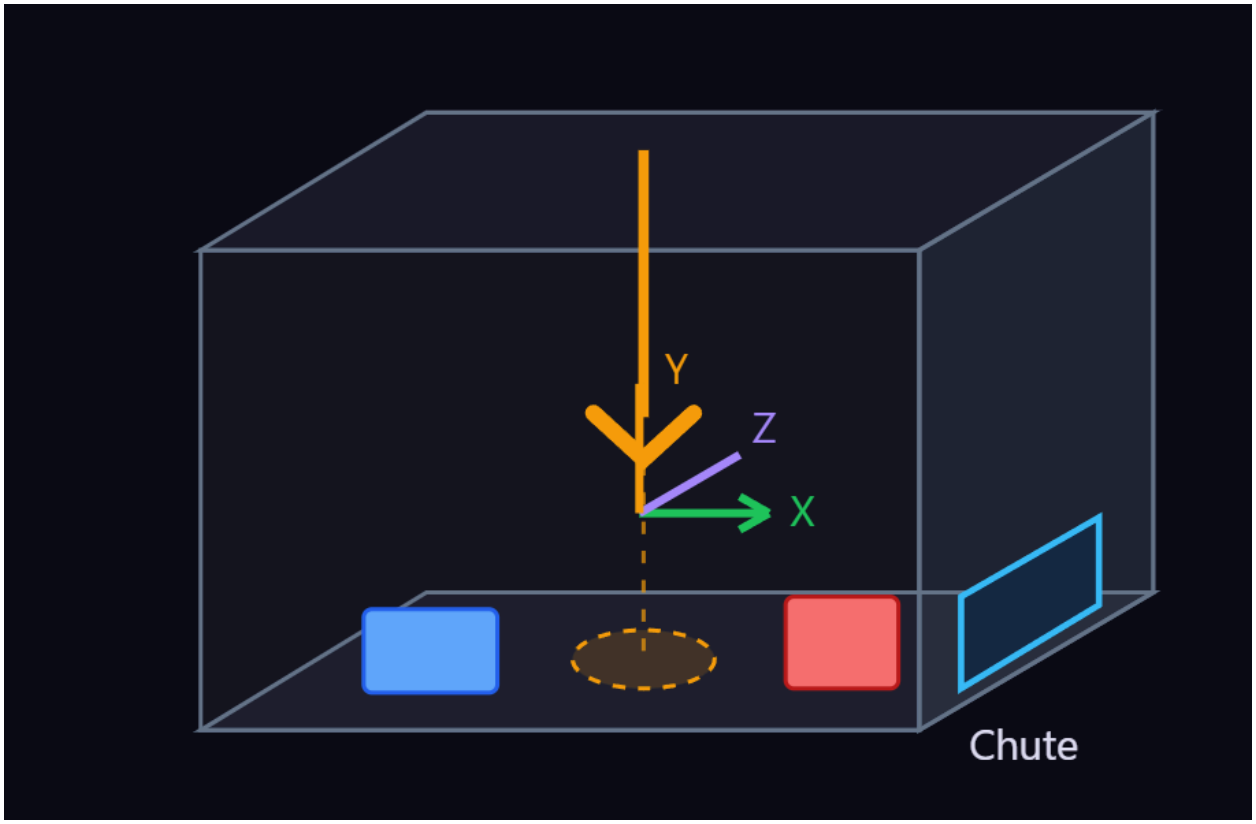


Figure 2. Proposed three-dimensional maintenance environment showing the claw, target snack, obstacles, and dispensing chute.

Interface and Visual Design

The interface is designed to resemble part of the vending machine rather than a separate modern game overlay.

During purchasing, the interface displays ordinary commercial messages such as payment and selection instructions. During recovery, the interface changes to a more technical maintenance display showing the target item, remaining attempts, and relevant controls.

This distinction reinforces the transition between the ordinary exterior and the unusual interior.

List of Objects

Graphical Object	Description and Purpose
Vending Machine Cabinet	The primary object in the 2D scene. It contains the snack display, selection controls, coin slot, and dispensing chute.
Glass Display	A transparent-looking panel that shows the available snacks and serves as the location of the final reflection sequence.
Snack Packages	Chips, chocolate bars, and juice cartons represented using distinct colors and geometric shapes. They appear in both 2D and 3D environments.
Coin Slot	A clickable machine component used to initiate the purchasing process.
Selection Buttons	Interactive buttons that allow the player to choose the snack requested by the customer.

Customer Characters	Three simple geometric figures with different appearances and dialogue. They provide the requests that initiate each stage.
Street Environment	Buildings, windows, pavement, streetlamps, and other geometric decorations that establish the nighttime setting.
Mechanical Claw	The primary controllable object in the 3D scene. It includes a cable, claw head, and gripping components used to retrieve snacks.
Claw Position Marker	A visual indicator projected onto the compartment floor to help the player align the claw with a target.
Industrial Compartment	The three-dimensional interior containing walls, flooring, shelves, structural supports, and decorative machinery.
Fixed Obstacles	Snack packages and other solid objects that restrict movement and require careful navigation.
Moving Restocking Arm	A moving mechanical obstacle introduced in the third stage.
Delivery Chute	The destination where the player must release the recovered snack to complete the rescue.
Interface Panels	Graphical elements that display dialogue, purchase information, rescue status, pause options, and failure messages.

Customer Reflection	A visual representation displayed in the machine's glass, used primarily for the final narrative sequence.

OpenGL Primitives Used

PLEASE INSERT COIN is built entirely from OpenGL primitives; no external models or textures are loaded. Every object in the game, from the street buildings to the claw prongs, is a cluster of vertices submitted either with glBegin()/glEnd() or through a vertex array. Seven primitive types are used in the source code. GL_POLYGON, GL_TRIANGLE_STRIP and GL_QUAD_STRIP are covered in the module but are not used in this project.

Primitive	Where it is used (functions)	What it renders	Module 02
GL_POINTS	drawSky()	The night-sky stars, drawn with glPointSize(); their alpha changes slowly so they twinkle.	pp. 6-7
GL_LINES	drawClaw3D(), drawMarker3D(), drawChute3D(), outlineBox3D(); also gRail2D and gGrid3D through glDrawArrays()	Claw cable, plumb line under the claw, chute light shafts, target outline edges, shelf rails and dividers in the machine glass, and the interior floor grid.	pp. 8-9
GL_LINE_STRIP	drawCustomerBody()	The customers' mouths (a flat line, or a wide smile for the third customer).	p. 10
GL_LINE_LOOP	strokeRect(), outlineBox3D(), drawMarker3D(), drawChute3D()	Borders of panels and buttons, the target highlight box, the claw-position marker, and the chute rim that equals the acceptance rectangle.	p. 11
GL_TRIANGLES	fillTri(), drawSnack3D(), drawClaw3D(), drawChute3D();	Crimped chip-bag tops, juice-carton roofs, chocolate wrapper ends, the tie and collar, glass glare, the three claw prongs, the chute arrow, and	pp. 20-21

	glDrawElements() in drawCuboid()	every cuboid in the 3D scene (12 triangles each).	
GL_TRIANGLE_FAN	fillEllipse(), drawStreetlamp()	Customers' heads (a fan around a centre vertex) and the translucent light pool under each streetlamp.	pp. 26-27
GL_QUADS	fillRect(), drawSky(), drawInteriorShell(), drawChute3D(), drawArm3D(), drawMarker3D()	Almost all flat shapes: buildings, windows, pavement, the machine cabinet, panels, buttons, the sky gradient, interior floor stripes, the chute pad, arm hazard stripes and the marker fill.	pp. 28-29

Table 11.1. OpenGL primitives used in the source code.

Colour attributes. Module 02 treats colour as a vertex attribute in RGBA mode (pp. 32-38). The game keeps its colours in a shared palette of `Col` structures (for example `PAL_AMBER` and `PAL_TEAL`) and applies them with `setCol()`, which calls `glColor4f()`. Per-vertex colours are used in two places: the sky gradient and the streetlamp light pool, where the centre vertex has an alpha of about 0.3 and the rim vertices an alpha of 0. In the indexed cuboid mesh, colours come from a colour array instead (Section 20).

```
static void fillRect(float x0, float y0, float x1, float y1) {
    glBegin(GL_QUADS);
    glVertex2f(x0, y0); glVertex2f(x1, y0); glVertex2f(x1, y1); glVertex2f(x0, y1);
    glEnd();
}
```

Listing 11.1. `fillRect()`, the `GL_QUADS` helper behind most 2D shapes.

Composite objects are built from these primitives so that no single triangle stands in for an object. A customer, for example, is two leg quads, a torso quad, two arm quads, a triangle fan for the head, hair and eye quads, and a line strip for the mouth.

Program Flow

The program is an explicit state machine. A single `GameState` value (`G.state`) decides which scene is drawn and which inputs are accepted. Pause, help and confirmation screens are

not separate states; they are an Overlay value stored beside the state so that a paused rescue can resume exactly where it stopped.

Stages of play

1. **Title (`TITLE`).** The quiet nighttime street and the glowing machine are shown, with the title and two options: INSERT COIN and HOW TO PLAY. HOW TO PLAY opens a purchase-only help panel. Starting calls startGame(), which resets everything and enters CUSTOMER_INTRO.
2. **Customer introduction (`CUSTOMER_INTRO`).** The customer for the current stage stands beside the machine and states a request in the dialogue band. Continuing moves to MACHINE_2D with no coin credit.
3. **Purchasing (`MACHINE_2D`).** The player clicks the coin slot (insertCoin()), then clicks a snack button (selectSnack()). A wrong choice keeps the credit and shows feedback; the correct choice consumes the credit.
4. **Malfunction (`TRANSITION_TO_3D`).** A timer-driven sequence of about 2.8 seconds: the snack jams in the glass with the message DISPENSING INTERRUPTED (1.0 s), a brief warning border pulses (0.7 s), then the camera shifts into the interior (1.1 s).
5. **3D rescue (`RESCUE_3D`).** The player moves the claw, lowers it, carries the target to the lit chute and releases it. ClawState tracks the substate (IDLE_EMPTY, DESCENDING, ASCENDING_EMPTY, ASCENDING_WITH_SNACK, CARRYING, RESOLVING_DROP).
6. **Failure or success.** A miss or an invalid drop calls resolveFailure(), which costs one attempt and returns claw and target to a safe layout. When no attempts remain, the state becomes GAME_OVER (RECOVERY UNSUCCESSFUL) and Retry calls retryStage(). A valid drop calls deliverSuccess(), entering TRANSITION_TO_2D.
7. **Result (`STAGE_RESULT`).** ITEM RECOVERED is shown. One acknowledgment collects the item and reveals the customer's reaction; a second continues.
8. **Stage advancement.** For stages 1 and 2, initializeStage(stageIndex + 1) loads the next customer and returns to CUSTOMER_INTRO. After stage 3, the state becomes REFLECTION_OUTRO.
9. **Final reflection (`REFLECTION_OUTRO`).** In four timed phases (customer exit 0.9 s, linger 1.4 s, turn 1.1 s, flicker and vanish 0.6 s) the last customer leaves, their faint

reflection remains in the machine glass, turns toward the viewer and disappears during a light flicker. The program then calls returnToTitle(), and the ordinary PLEASE INSERT COIN screen returns. No text explains the sequence.

Program-flow diagram

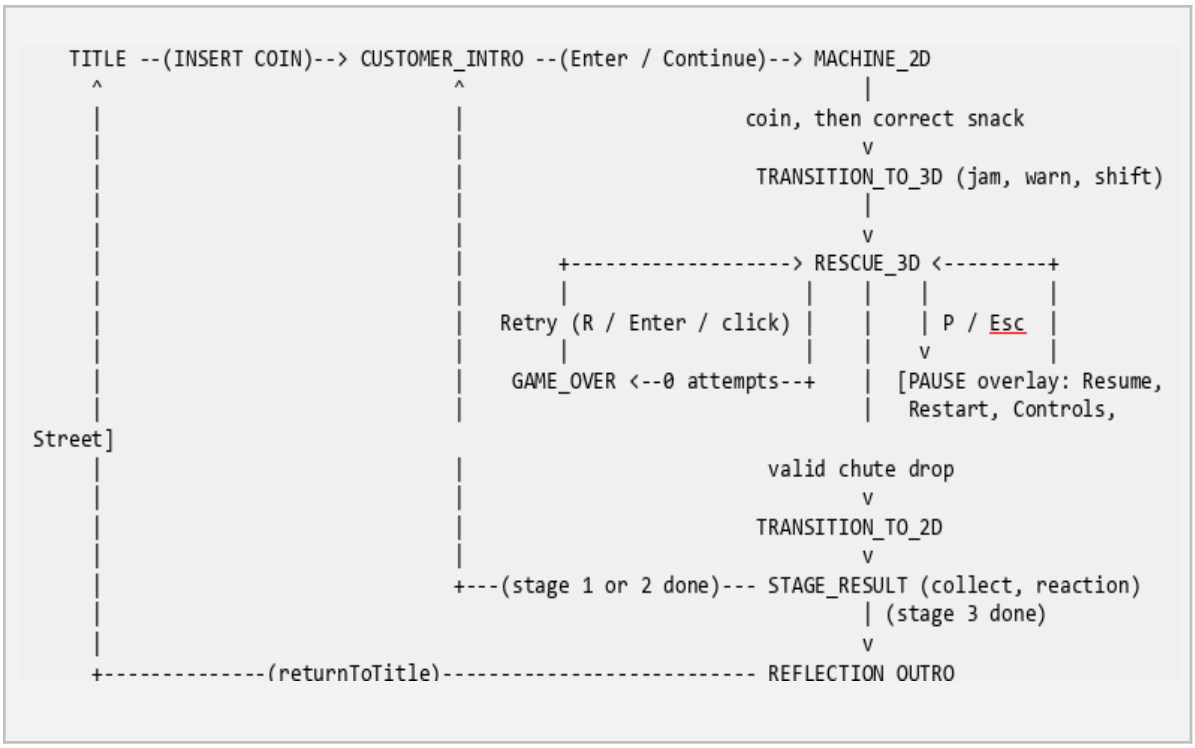


Figure 12.1. State diagram of `GameState` (text specification; replace with a drawn diagram if desired). Return to Street in the pause menu also calls `returnToTitle()`.

Timing

A single timer chain, timerUpdate(), drives everything. Each call reads glutGet(GLUT_ELAPSED_TIME), clamps the delta to 50 ms, advances presentation clocks, and runs gameplay in fixed 10 ms steps (FIXED_STEP) only during an unpaused rescue. display() only draws; it never changes game state.

Controls

Keyboard controls are case-insensitive for letters. Key-repeat is disabled with glutIgnoreKeyRepeat(1), so every key-down event is a fresh press. Mouse actions use the left button only and act on button-down.

Input	Function	Available in
Mouse: left click on INSERT COIN / HOW TO PLAY	Start a new game / open purchase help	TITLE
Up / Down (or W / S), Enter	Move focus between the two title options; activate the focused one	TITLE
Mouse: CONTINUE button, or Enter	Move from the customer request to the purchase view	CUSTOMER_INTRO
Mouse: coin slot	Insert one coin (extra coins are ignored)	MACHINE_2D
Mouse: snack button (CHIPS, CHOC, JUICE)	Select a snack; needs a coin; only the requested snack proceeds	MACHINE_2D
W A S D or arrow keys (hold)	Move the claw: X is left/right, Z is depth. W/Up moves deeper, S/Down toward the front	RESCUE_3D while IDLE_EMPTY or CARRYING
Space	Context-sensitive: lowers the claw to grab when empty; releases the carried snack when carrying	RESCUE_3D, IDLE_EMPTY / CARRYING
P or Esc	Pause (opens the pause menu); P/Esc again resumes	RESCUE_3D (any claw substate)
Up / Down, Enter, mouse hover and click	Navigate and activate RESUME, REINITIALIZE, CONTROLS, RETURN TO STREET	Pause menu
Left / Right / Up / Down, Enter, Esc, mouse	Choose CANCEL or CONFIRM (Cancel is the default); Esc cancels	Restart / Return-to-Street confirmation
Enter, Esc, or BACK button	Close help; from rescue help, return to the pause menu	Help panels
Mouse: delivered snack in the chute, or Enter	Collect the item (first acknowledgment)	STAGE_RESULT (collection phase)
Mouse: CONTINUE, or Enter	Continue after the customer's reaction (second acknowledgment)	STAGE_RESULT (reaction phase)
R, Enter, or mouse click on RETRY	Retry the current stage (restores all attempts)	GAME_OVER only

(none)	No input is accepted	TRANSITION_TO_3D, TRANSITION_TO_2D, REFLECTION_OUTRO

Table 13.1. Controls by input and application state.

Context-sensitive behaviour

- **Space** has two meanings, chosen by ClawState: requestGrabOrRelease() starts a descent from IDLE_EMPTY and a release from CARRYING, and ignores Space during any other substate.
- **R** retries only on the game-over screen. During active rescue it does nothing, so the player cannot refill attempts mid-action. Restarting a stage during play is possible only through the confirmed REINITIALIZE option in the pause menu.
- **Esc** means pause during rescue, "back" inside overlays, and nothing elsewhere; it does not quit the program.
- **Overlays take priority.** handleKeyDown() and handleClick() check G.overlay first, so when a menu is open it receives the input exclusively.
- **On-screen hints** (for example SPACE: LOWER CLAW or SPACE: RELEASE) are drawn from the same control-label constants used by the help panel, so the displayed controls match the implemented ones.

Explanation of Functions

The whole program is one .cpp file organized into ten commented sections, each a group of small functions with one responsibility. This project presents a GLUT program as a set of functions registered as callbacks but divides the work by responsibility: authoritative state is changed only by gameplay and input helpers, while drawing functions only read it.

Category	Representative functions	Responsibility
Initialization	main(), buildStaticGeometry(), initializeNewGame(), initializeStage()	main() calls glutInit(), creates the window, builds the static vertex arrays, registers callbacks and starts the timer chain. initializeStage() sets attempts, clears the coin and cue clocks and calls resetRescueLayout().

State transitions	transitionTo(), startGame(), returnToTitle(), acknowledgeResult()	transitionTo() sets the new state, resets its clock and clears held keys so a state change cannot leave a stuck key.
Gameplay logic	insertCoin(), selectSnack(), requestGrabOrRelease(), stepRescue(), moveClaw(), updateArm()	Purchasing rules, the claw state machine and fixed-step simulation.
Collision detection	footprintBlocked(), rectOverlap(), targetInGrabRange(), inChute()	Boundary and obstacle tests, the forgiving grab tolerance and the chute acceptance rectangle.
Resetting	resetRescueLayout(), resolveFailure(), retryStage()	Safe reset after a failed attempt, the single place attempts are deducted, and a full same-stage retry.
Rendering (2D)	drawStreet2D(), drawMachine2D(), drawCustomer(), drawReflection()	Layered street, machine, characters and reflection.
Rendering (3D)	begin3D(), drawScene3D(), drawInteriorShell(), drawClaw3D(), drawMarker3D()	Fixed oblique camera and the interior built from the indexed cuboid.
Primitives and text helpers	fillRect(), fillTri(), fillEllipse(), drawText(), drawWrapped()	Reusable drawing building blocks.
Callbacks	display(), reshape(), keyboardDown(), specialDown(), mouse(), timerUpdate()	Thin adapters that validate input and call one named action.

Table 14.1. Function organization.

Initialization. resetRescueLayout() is a small example of a function with one clear job. It puts the claw at the stage's start position, empties it, restores the target to its stage position and resets the arm and held keys, but it does not touch the remaining attempts or the purchase state:

```
static void resetRescueLayout() {
    const StageData& s = stage();
    C.x = s.startX; C.z = s.startZ; C.y = CLAW_TOP_Y;
    C.state = IDLE_EMPTY; C.carrying = false; C.missPending = false;
    C.dropSuccess = false; C.dropT = 0.0f;
    C.snackX = s.tx; C.snackZ = s.tz; C.snackY = SNACK_H[s.target] * 0.5f;
    C.snackFallFromY = C.snackY;
    resetArm();
    clearHeldKeys();
    gAccum = 0.0f;
}
```

Listing 14.1. `resetRescueLayout()`.

Collision detection. footprintBlocked() tests a rectangle against every fixed package and, in stage 3, against the moving arm. Each package type has its own half-extents (SNACK_HX, SNACK_HZ) that match what is drawn, including the chocolate wrapper ends:

```
static bool footprintBlocked(float x, float z, float hx, float hz) {
    const StageData& s = stage();
    for (int i = 0; i < s.obsCount; ++i) {
        int t = s.obs[i].type;
        if (rectOverlap(x, z, hx, hz, s.obs[i].x, s.obs[i].z, SNACK_HX[t], SNACK_HZ[t])) return true;
    }
    if (s.hasArm && rectOverlap(x, z, hx, hz, K.armX, ARM_Z, ARM_HX, ARM_HZ)) return true;
    return false;
}
```

Listing 14.2. `footprintBlocked()`.

State transitions and reset. All failures go through resolveFailure(), which is guarded by G.cycleResolved so one grab-and-drop cycle can cost at most one attempt. When the last attempt is spent, it enters GAME_OVER before resetting, so the failed pose stays on the screen behind the message.

```
static void resolveFailure(FailReason why) {
    if (G.cycleResolved) return;              // one cycle, one charge
    G.cycleResolved = true;
    if (G.attempts > 0) G.attempts--;
    ...
    if (G.attempts <= 0) { transitionTo(GAME_OVER); return; }   // keep the failed pose as the backdrop
    resetRescueLayout();                      // safe layout, attempts preserved
}
```

Listing 14.3. `resolveFailure()` (message-formatting lines omitted).

Explanation of Conditions

Conditions decide which action is legal in the current state and what the result is. switch statements are used where one variable (a state or substate) selects among several branches, and if/else where a yes-or-no test is needed.

Purchasing

selectSnack() is a chain of guard conditions, checked from the most basic requirement to the most specific. Only the last branch has a lasting effect:

```
static void selectSnack(int type) {
    if (G.state != MACHINE_2D) return;
    if (!G.hasCoin) { setFeedback("INSERT COIN FIRST"); return; }
    if ((SnackType)type != stage().target) { setFeedback("PLEASE CHECK YOUR SELECTION"); return; }
    G.hasCoin = false;                  // correct pick consumes the credit
    resetRescueLayout();
    transitionTo(TRANSITION_TO_3D);
}
```

Listing 15.1. Purchase conditions.

Without a coin, the click gives feedback and does nothing else. A wrong snack keeps the coin and costs no attempt. A correct snack consumes the coin and starts the transition once.

Game states and input

handleKeyDown() first checks G.overlay != OV_NONE, so an open menu consumes the key. It then uses a switch on G.state, and each case accepts only the keys valid for that state. The transition and outro states fall through to a default that ignores all input.

Claw behaviour

stepRescue() is a switch on ClawState. In IDLE_EMPTY and CARRYING, horizontal movement is allowed; in the other substates it is not. The grab decision is a condition on the grab tolerance, with equality accepted:

```
case DESCENDING:
  C.y -= DESCEND_SPD * dt;
  if (C.y <= clawBottomY()) {
    C.y = clawBottomY();
if (targetInGrabRange()) { C.carrying = true; C.state = ASCENDING_WITH_SNACK; }
else { C.missPending = true; C.state = ASCENDING_EMPTY; }
}
break;
```

Listing 15.2. Grab decision inside `stepRescue()`.

Collision outcomes

moveClaw() first clamps the proposed position to the compartment walls, then accepts the move only if footprintBlocked() returns false. X and Z are tested separately, so the claw slides along an obstacle instead of sticking to it. On release, inChute() is evaluated once, at the moment of release, and the result is stored in C.dropSuccess; the fall animation cannot change it.

Attempts and progression

- resolveFailure() decrements G.attempts only once per cycle and switches to GAME_OVER when attempts reach zero.
- acknowledgeResult() uses G.resultPhase: phase 0 collects the item; phase 1 advances. The condition G.stageIndex < 2 chooses between loading the next stage and starting the outro, which also keeps the stage index within 0-2.
- updateArm() reverses the arm's direction when its next step would overlap the claw or the carried snack, instead of pushing through.

Explanation of Loops

Loops generate repeated geometry from a few parameters and iterate over small arrays of game data. No loop in the program blocks or waits; animation is advanced by the timer callback, not by a loop with a delay.

Function	Loop	Purpose
drawBuilding()	Nested for over columns i and rows j	Places window rectangles in a grid; lit or unlit is chosen by windowLit(i, j), so the pattern is fixed.

drawStreet2D()	for over 8 buildings	Draws the skyline from three small arrays of positions, widths and heights.
drawSky()	for over 40 stars	Positions each star from its index and varies its alpha over time.
drawPavement()	for over 20 slabs	Draws the pavement seams.
drawMachine2D()	Nested for over 3 shelves and 4 slots	Places snacks on the shelves; (k + s) % 3 selects the snack type. Another loop draws the three buttons.
buildStaticGeometry()	Several for loops	Fills gRail2D, gGrid3D and the 36 cuboid indices once at start-up.
drawInteriorShell()	for loops over stripes, pipes, vents, bolts, columns and racks	Generates the oversized industrial scenery from repeated cuboids.
drawScene3D()	for over S.obsCount	Draws every fixed package of the current stage from the StageData array.
footprintBlocked()	for over the stage's obstacles	Tests the claw footprint against each package.
fillEllipse()	for over seg points	Generates the rim vertices of a triangle fan.
drawTextRaw(), textW()	for (; *s; ++s)	Outputs or measures a string one character at a time (Section 19).
wrapText()	while (*p) over words	Breaks text into lines that fit a panel.
timerUpdate()	while (gAccum >= FIXED_STEP && n < MAX_CATCHUP)	Runs gameplay in fixed steps with a bounded catch-up.
updateOutro()	while over elapsed phase durations	Crosses outro phase boundaries in order, once each, even after a long frame.
handleClick()	for over buttons and menu options	Hit-tests each shared rectangle.

Table 16.1. Representative loops.

```
// the three shelves of the 2D machine, four slots each
for (int s = 0; s < 3; ++s)
for (int k = 0; k < 4; ++k) {
int type = (k + s) % 3;
drawSnack2D(type, R_GLASS.x0 + 4 + k * 16.0f, shelfY[s] + 1, 11, type == SNACK_CHOC ? 12 : 17);
}
```

Listing 16.1. Nested loops in `drawMachine2D()` (jam-slot test omitted).

The fixed-step loop in timerUpdate() deserves a note. Elapsed time is accumulated in gAccum, and one stepRescue(FIXED_STEP) is run for each 10 ms accumulated. The loop is capped at MAX_CATCHUP steps, and any further backlog is discarded, so a stalled window cannot make the claw jump far on the next frame.

Explanation of Keyboard Callback

glutKeyboardFunc() is the callback that receives ordinary ASCII key presses. The game registers four keyboard-related callbacks in main():

```
glutIgnoreKeyRepeat(1); // each key-down is a fresh press
glutKeyboardFunc(keyboardDown); // ASCII keys pressed
glutKeyboardUpFunc(keyboardUp); // ASCII keys released
glutSpecialFunc(specialDown); // arrow keys pressed
glutSpecialUpFunc(specialUp); // arrow keys released
```

Listing 17.1. Keyboard callback registration in `main()`.

The key-up and special-key callbacks go beyond the basic example in the module; they are needed because the claw moves while a key is held, and the arrow keys are not ASCII characters.

Two kinds of input

The callbacks separate input into held keys and one-shot actions.

- **Held keys** (movement). Each physical binding has its own flag in gHeld[] (H_A, H_D, H_W, H_S, H_LEFT, H_RIGHT, H_UP, H_DOWN). Separate flags mean that releasing A does not cancel a still-held Left arrow. moveClaw() reads the flags every fixed step: opposite directions cancel, and diagonal speed is scaled by 0.7071 so it matches straight movement. Movement stops the instant the key is released, because there is no inertia.

- **One-shot actions** (Space, P/Esc, Enter, R). The callback converts the key to a LogicalKey (LK_ACTION, LK_PAUSE, LK_ESC, LK_ENTER, LK_RETRY, and the four directions) and calls handleKeyDown() once. Because key-repeat is disabled, holding Space cannot start more than one grab, and holding R or Enter cannot retry repeatedly.

```
static void keyboardDown(unsigned char key, int, int) {
if (key >= 'A' && key <= 'Z') key = (unsigned char)(key - 'A' + 'a');   // case-insensitive
LogicalKey lk = LK_NONE; int held = -1;
switch (key) {
case 'w': lk = LK_UP;            held = H_W; break;
case 's': lk = LK_DOWN;  held = H_S; break;
case 'a': lk = LK_LEFT;  held = H_A; break;
case 'd': lk = LK_RIGHT; held = H_D; break;
case ' ': lk = LK_ACTION; break;
case 'p': lk = LK_PAUSE; break;
case 27:  lk = LK_ESC; break;
case 13:  lk = LK_ENTER; break;
case 'r': lk = LK_RETRY; break;
}
if (held >= 0 && movementAccepted()) setHeld(held, true);
if (lk != LK_NONE) handleKeyDown(lk);
}
```

Listing 17.2. `keyboardDown()`.

State restrictions

The callback itself contains no game rules. movementAccepted() allows a held movement flag to be recorded only while the game is in RESCUE_3D, no overlay is open, and the claw is IDLE_EMPTY or CARRYING. handleKeyDown() then checks the overlay first and the game state second (Section 15.2). Starting a grab or a release calls clearHeldKeys(), so movement pressed during a descent is not stored and applied afterwards.

Pause and retry

- **Pause.** In RESCUE_3D, P or Esc calls pauseRescue(), which sets G.overlay = OV_PAUSE and clears the held keys. Because timerUpdate() runs gameplay only when the overlay is OV_NONE, the claw, the carried snack and the arm all freeze. On resumeRescue(), the

held keys are cleared again, the accumulator is zeroed and gLastMs is refreshed, so no paused time is applied as movement. A fresh key press is needed to move again.

- **Retry.** In GAME_OVER, R or Enter calls retryStage(). R is ignored in every other state.
- **Focus.** visibilityFunc() clears held keys when the window is hidden, and on Windows the timer clears them when the game window is no longer the foreground window. Pointer enter/leave events are deliberately not used for this, because they do not indicate keyboard focus.

Menu navigation

handleOverlayKey() receives the same logical keys. In the pause menu, Up and Down change G.menuSel with wrap-around using menuMove(), Enter calls activatePauseOption(), and P or Esc resumes. Confirmation dialogs toggle G.confSel with the arrow keys and default to CANCEL.

Explanation of Mouse Callback

The game registers glutMouseFunc(mouse) for button events and glutPassiveMotionFunc(passiveMotion) so that menu options highlight under the pointer. The mouse callback accepts only a left-button press, so a click performs exactly one action:

```
static void mouse(int button, int state, int mx, int my) {
    if (button != GLUT_LEFT_BUTTON || state != GLUT_DOWN) return;   // one click = one action
    float lx, ly;
    if (!mouseToLogical(mx, my, lx, ly)) return;                    // letterbox bars rejected
    handleClick(lx, ly);
    glutPostRedisplay();
}
```

Listing 18.1. The mouse callback.

Coordinate conversion

GLUT reports the mouse in window pixels with the origin at the top-left. The game draws in a 320 x 180 logical space with the origin at the bottom-left, centred in the window as a 16:9 rectangle with black letterbox bars. computeViewport() calculates the scale and the content rectangle (gVP), and mouseToLogical() flips the Y axis and maps the pixel into logical units. A click that lands in a bar returns false and is ignored.

```
static bool mouseToLogical(int mx, int my, float& lx, float& ly) {
                                    float px = (float)mx;
float py = (float)(gWinH - my);                    // flip to bottom-left origin
                                    lx = (px - gVP.x) / gVP.scale;
                                    ly = (py - gVP.y) / gVP.scale;
  return lx >= 0.0f && lx <= LW && ly >= 0.0f && ly <= LH;   // reject bars
                                    }
```

Listing 18.2. Mouse-to-logical conversion.

Hit testing

Every clickable control is a named Rect (for example R_COIN, R_BTN[3], R_CHUTE2D, R_PAUSE_OPT[4]). The drawing code and inRect() use the same rectangles, so what is drawn is what can be clicked, at any window size. handleClick() checks the overlay first, then branches on the game state:

- **`MACHINE_2D`:** a click in R_COIN calls insertCoin(); a loop over R_BTN[] calls selectSnack(i) for the button that was hit.
- **`STAGE_RESULT`:** during collection, a click on R_CHUTE2D (the delivered snack) calls acknowledgeResult(); during the reaction, R_CONTINUE does.
- **Menus:** R_PAUSE_OPT[], R_CONF_CANCEL, R_CONF_OK, R_HELP_BACK, R_TITLE_START, R_TITLE_HELP and R_RETRY activate the matching option.
- **Hover:** passiveMotion() updates the focus index (G.menuSel, G.confSel, G.titleSel) from the same rectangles so keyboard and mouse share one focus.

The callback only validates and dispatches. The game rules, such as requiring a coin before a selection, live in insertCoin() and selectSnack(), so the keyboard and mouse paths cannot disagree.

Explanation of Text Rendering

```
static void drawTextRaw(float x, float y, const char* s, void* font) {
                                    glRasterPos2f(x, y);
                                    for (; *s; ++s) glutBitmapCharacter(font, *s);
                                    }
static void drawText(float x, float y, const char* s, Col c, void* font, float a = 1.0f) {
                                    float o = 0.35f;
```

```
                setCol(PAL_INK, a); drawTextRaw(x + o, y - o, s, font);   // dark shadow
                setCol(c, a);           drawTextRaw(x, y, s, font);
                }
```

Listing 19.1. Raster-positioned text with a drop shadow.

Helpers

- **Colour.** The colour is set with glColor4f() before glRasterPos2f() is called. drawText() draws a dark copy offset by a fraction of a logical unit first, to keep text readable over any background.
- **Font selection.** fontSmall() returns GLUT_BITMAP_9_BY_15 when the window scale is at least 3 and GLUT_BITMAP_8_BY_13 otherwise; fontBig() returns GLUT_BITMAP_HELVETICA_18 or GLUT_BITMAP_HELVETICA_12. Bitmap fonts are fixed in screen pixels, so the choice keeps text readable on large and small windows.
- **Measuring and centring.** textW() sums glutBitmapWidth() for each character and divides by the scale to get logical units. drawTextCentered() uses it to centre a line on a point.
- **Fitting.** lineH() gives a line height in logical units, wrapText() breaks a string on spaces to fit a width, and drawWrapped() and textBlock() draw the wrapped lines and size a panel around them. drawButton() and drawDialogueBand() swap in a short label or prompt when the full one does not fit.

What the text displays

Text	Drawn by	Content
Title and help	drawTitleUI(), drawHelpPanel()	PLEASE INSERT COIN, INSERT COIN, HOW TO PLAY, and purchase-only instructions; rescue controls appear only in the paused-rescue help.
Purchase display	drawPurchaseHUD()	CREDIT: 0/1, SELECT ITEM, and feedback such as INSERT COIN FIRST.
Dialogue	drawDialogueBand()	Customer requests and reactions in the bottom band, plus prompts.

Rescue terminal	drawRescueHUD()	MAINTENANCE ACCESS, TARGET, ATTEMPTS, and the available action hints taken from the shared control labels.
Menus and results	drawPauseMenu(), drawConfirm(), drawResultUI(), drawGameOverUI()	Pause options, confirmations, ITEM RECOVERED, RECOVERY UNSUCCESSFUL.
On the machine	drawMachine2D()	The small display (INSERT COIN or SELECT ITEM) and the button labels; these are drawn only if they fit.

Table 19.1. Text surfaces.

No text anywhere in the game explains the customers or the ending.

Explanation of Vertex Arrays

Two draw paths are used in normal gameplay.

Path	Data	Call	Visible in the game
Non-indexed	gRail2D: 42 vertices (x, y); gGrid3D: 24 vertices (x, y, z)	glVertexPointer() + glDrawArrays(GL_LINES, 0, count)	Shelf rails and slot dividers in the machine glass; the floor grid of the interior.
Indexed	gCubePos: 24 vertex records (x, y, z); gCubeCol: 24 colours (r, g, b); gCubeIdx: 36 GLushort indices	glVertexPointer() + glColorPointer() + glDrawElements(GL_TRIANGLES, 36, GL_UNSIGNED_SHORT, gCubeIdx)	Every cuboid in the 3D scene: packages, walls, pipes, gantry, claw head and trolley, arm, chute frame.

Table 20.1. Vertex-array paths in PLEASE INSERT COIN.

Non-indexed arrays: `glDrawArrays()`

buildStaticGeometry() fills gRail2D once at start-up using loops. For each of three shelves it stores two horizontal lines (the lip and the underside) and five vertical dividers, each as a pair of vertices. drawRails2D() then draws them all with one call:

```
static void drawRails2D(Col c) {
glEnableClientState(GL_VERTEX_ARRAY);
glVertexPointer(2, GL_FLOAT, 0, gRail2D);        // 2 floats per vertex, tightly packed
setCol(c); glLineWidth(2.0f);
glDrawArrays(GL_LINES, 0, gRailCount);           // start at vertex 0, use gRailCount vertices
glDisableClientState(GL_VERTEX_ARRAY);
glLineWidth(1.0f);
}
```

Listing 20.1. `drawRails2D()` using `glVertexPointer()` and `glDrawArrays()`.

drawGrid3D() does the same with three floats per vertex for the interior floor grid. The size argument of glVertexPointer() is 2 for the 2D array and 3 for the 3D array, the type is GL_FLOAT, and the stride is 0 because the values are tightly packed (Module 04, p. 10).

Indexed mesh: `glDrawElements()`

The 3D scene is built from one reusable unit cuboid. Its mesh has 6 faces, and each face has 4 vertex records, so the position array holds 24 records. Although a cube has only 8 distinct corners, each corner belongs to three faces. Every face needs its own colour because the game uses flat face shading (the top face is brighter than the sides, the bottom darker). Since a vertex carries one colour, the corner is stored once per face. Each face is then drawn as two triangles using 6 indices, giving 36 indices for 12 triangles.

```
static const GLfloat gCubePos[24 * 3] = {        // 6 faces x 4 corners, face-local records
// +X
.5f,-.5f,-.5f,  .5f, .5f,-.5f,  .5f, .5f, .5f,  .5f,-.5f, .5f,
// -X
-.5f,-.5f, .5f, -.5f, .5f, .5f, -.5f, .5f,-.5f, -.5f,-.5f,-.5f,
// +Y (top) ... -Y ... +Z ... -Z (front, faces the camera)
...
};
static const GLfloat gFaceShade[6] = { 0.80f, 0.62f, 1.00f, 0.42f, 0.68f, 0.90f };

for (int f = 0; f < 6; ++f) {                    // in buildStaticGeometry()
GLushort b = (GLushort)(f * 4), *p = &gCubeIdx[f * 6];
p[0] = b; p[1] = b + 1; p[2] = b + 2;      // triangle 1 of the face
```

```
                p[3] = b; p[4] = b + 2; p[5] = b + 3;      // triangle 2 of the face
            }
```

Listing 20.2. Cuboid vertex records, face shades and index generation (middle faces abbreviated).

```
static void drawCuboid(float cx, float cy, float cz, float sx, float sy, float sz, Col c) {
    for (int i = 0; i < 24; ++i) {                  // tint the per-face shade with the object colour
        float sh = gFaceShade[i / 4];
        gCubeCol[i*3] = c.r * sh; gCubeCol[i*3+1] = c.g * sh; gCubeCol[i*3+2] = c.b * sh;
    }
    glPushMatrix();
    glTranslatef(cx, cy, cz); glScalef(sx, sy, sz);
    glEnableClientState(GL_VERTEX_ARRAY); glEnableClientState(GL_COLOR_ARRAY);
    glVertexPointer(3, GL_FLOAT, 0, gCubePos);
    glColorPointer(3, GL_FLOAT, 0, gCubeCol);
    glDrawElements(GL_TRIANGLES, 36, GL_UNSIGNED_SHORT, gCubeIdx);
    glDisableClientState(GL_COLOR_ARRAY); glDisableClientState(GL_VERTEX_ARRAY);
    glPopMatrix();
}
```

Listing 20.3. `drawCuboid()`: one mesh, many objects.

drawCuboid() is the reuse mechanism. The mesh is defined once at unit size around the origin; glTranslatef() and glScalef() position and size each instance, and the colour array is refilled with the object's colour multiplied by the face shades. Walls, pipes, bolts, shelves, packages, the claw head, the arm and the gantry rails are all this same mesh. The count passed to glDrawElements() is the number of indices (36), not the number of triangles, and the index type GLushort matches GL_UNSIGNED_SHORT. All indices are below 24, the number of vertex records. Because the arrays are static, they remain valid for the whole draw call, and client states are disabled after each draw so later immediate-mode drawing and text are unaffected.

The five required questions

The assignment asks for five explanations. They are answered here for the implemented data.

10. **What data is stored?** For the indexed mesh: 24 vertex positions (72 floats), a parallel colour array of 24 RGB values (72 floats) and 36 GLushort indices. For the non-indexed arrays: 42 two-float vertices for the 2D shelf rails and 24 three-float vertices for the 3D floor grid.

11. **Why use an array?** The same shapes are drawn many times per frame. Storing their vertices once in memory and pointing OpenGL at them removes the need to call glVertex*() for every vertex of every object each frame, as in the immediate-mode example in Module 04 (p. 6). It also keeps the geometry in one place, where it can be changed once.
12. **What does each vertex represent?** In gRail2D, each vertex is one end of a shelf-rail or divider segment in logical 2D space. In gGrid3D, each is one end of a floor-grid line in world units. In gCubePos, each record is one corner of one cuboid face in unit-cube space; the same corner appears once per face it belongs to, with its face's shade colour.
13. **Which primitive is rendered?** GL_LINES for the rails and grid (two vertices per segment) and GL_TRIANGLES for the cuboid (three indices per triangle, two triangles per face, 12 per cuboid).
14. **What is the advantage over repeatedly specifying the same data?** A cuboid drawn immediately would need 36 glVertex3f() calls plus colour calls every time. With the indexed mesh, the vertex and index data exist once, and each additional object costs one transform, one colour fill and one glDrawElements() call. Every cuboid in the scene shares the one definition. I did not benchmark the difference; the advantage claimed here is the reduction in repeated data specification described in the module, not a measured speed gain.

What an index is, and how `glDrawElements()` differs from `glDrawArrays()`. An index is the position of a vertex record in the vertex array. glDrawArrays() walks the vertex array in order, so each triangle must have its own consecutive vertices. glDrawElements() walks the index list and fetches vertices by number, so one record can be referenced by several triangles. In this mesh, the two triangles of each face reuse two of the face's four records (the indices b and b + 2). Sharing across faces is not possible here because faces have different shade colours, which is why the mesh has 24 records instead of 8.

Creative Feature

The main creative feature of PLEASE INSERT COIN is the contrast between an ordinary 2D purchase and a sudden, surreal 3D maintenance task, supported by a small mystery that is told only through visual details. The following features are implemented in the source code.

- **Unexpected 2D-to-3D transition.** After a correct purchase, TRANSITION_TO_3D runs a timed sequence: the snack visibly jams in the glass (with a shaking motion), a short warning border pulses, and begin3D() interpolates the camera from a position close to the machine to the final elevated oblique view while a warm flash fades. The view changes from an orthographic 2D scene to a perspective 3D scene.
- **Mechanical claw recovery.** The player moves the claw across the width (X) and depth (Z) of the interior, lowers it, carries the target and releases it over a lit chute. The claw has no inertia, the grab tolerance is forgiving (GRAB_TOL_X, GRAB_TOL_Z), and a carried snack never slips. A yellow marker under the claw and a pulsing outline on the target help judge depth.
- **Collision detection.** The claw, and the snack it carries, collide with the compartment walls, the fixed packages and the moving arm using the real footprint of each package type. Movement is resolved one axis at a time, so the claw slides along obstacles.
- **Limited attempts and recovery.** Each stage allows START_ATTEMPTS (3) failed grabs or invalid drops. There is no countdown. After the last attempt, the game over screen offers a same-stage retry.
- **Stage progression.** Three stages reuse the same controls and scene but change the customer, the target snack and the layout: an open approach with chips, a tighter arrangement with the chocolate bar, and the juice carton behind a moving restocking arm that reverses instead of pushing through the claw.
- **Environmental storytelling.** The customers become gradually odd through brief dialogue and small machine disturbances that are computed from the scene clock by computeCues(): a display dip and indicator blink in stage 1, irregular selection-light pulses and a recurring three-lamp pattern in stage 2, and an indicator flutter with nearby lights dimming in stage 3. The cues change only brightness, never gameplay. The ending (REFLECTION_OUTRO) shows the last customer's reflection lingering in the glass, turning toward the viewer and vanishing during a flicker. No text states what the customers are.

Planned or not implemented

The handoff describes several enhancements that are not part of the current source:

- Rendering the world into a low-resolution framebuffer (320 x 180 or 640 x 360) with nearest-neighbour enlargement was not implemented. The game draws directly to the window; the pixel-art look comes from grid-aligned shapes and the limited palette, so uniform pixelation is not claimed.
- Optional fog was not enabled.
- The claw prongs are drawn open or closed only; there is no animated gripping motion.
- There is no sound, scoring, extra stages or alternate endings, which were excluded from the approved scope.

Important Code Sections

Single timer chain and fixed-step gameplay

```
static void timerUpdate(int) {
int now = glutGet(GLUT_ELAPSED_TIME);
float dt = (now - gLastMs) / 1000.0f;
gLastMs = now;
if (dt > MAX_FRAME_DELTA) dt = MAX_FRAME_DELTA;                // long stalls never teleport anything
updatePresentation(dt);
if (G.state == RESCUE_3D && G.overlay == OV_NONE) {
gAccum += dt;
int n = 0;
while (gAccum >= FIXED_STEP && n < MAX_CATCHUP) {
gAccum -= FIXED_STEP; ++n;
stepRescue(FIXED_STEP);
if (G.state != RESCUE_3D) break;                // success/failure changed state
}
if (n >= MAX_CATCHUP) gAccum = 0.0f;                    // drop excess backlog
} else gAccum = 0.0f;                                // paused: no accumulated jump
glutPostRedisplay();
glutTimerFunc(TIMER_MS, timerUpdate, 0);                // exactly one next callback
}
```

Listing 22.1. `timerUpdate()` (Windows focus check omitted).

This is the only update path. Gameplay time advances only when the rescue is active and no overlay is open, which is what makes pause freeze the scene. display() never advances

any clock. Module 03 describes glutTimerFunc() (p. 30); the game re-registers the timer at the end of each call to form one chain.

Fixed oblique camera

```
static void begin3D(float eyeBlend) {
    applyContentViewport();
    glEnable(GL_DEPTH_TEST); glDepthFunc(GL_LEQUAL); glDepthMask(GL_TRUE);
    glDisable(GL_LIGHTING); glDisable(GL_TEXTURE_2D); glDisable(GL_FOG); glDisable(GL_BLEND);
    glMatrixMode(GL_PROJECTION); glLoadIdentity();
    gluPerspective(48.0, LW / LH, 0.5, 120.0);
    glMatrixMode(GL_MODELVIEW);  glLoadIdentity();
    float e = smooth01(eyeBlend);                    // 0 = close to the glass, 1 = rescue camera
    gluLookAt(0.0f, lerpf(3.0f, 9.2f, e), lerpf(-3.0f, -8.8f, e),
              0.0f, lerpf(2.5f, 1.6f, e), lerpf(3.0f, 4.0f, e),
              0.0f, 1.0f, 0.0f);
    glScalef(-1.0f, 1.0f, 1.0f);                     // looking along +Z mirrors X; undo so +X = screen right
}
```

Listing 22.2. `begin3D()`.

Each drawing pass sets its own projection, depth, blending and client-array state, so no scene depends on what the previous scene left behind. During the rescue, eyeBlend is 1 and the camera is fixed; during the reveal it moves from 0 to 1. The final glScalef(-1, 1, 1) corrects a mirrored X axis so that the D key and the right arrow move the claw to the right on the screen.

Grab and release as one context-sensitive action

```
static void requestGrabOrRelease() {
    if (G.state != RESCUE_3D || G.overlay != OV_NONE) return;
    if (C.state == IDLE_EMPTY) {
        clearHeldKeys();                             // motion input is ignored during the action
        G.cycleResolved = false;                     // a new grab cycle begins
        C.missPending = false;
        C.state = DESCENDING;
    } else if (C.state == CARRYING) {
        clearHeldKeys();
        C.snackX = C.x; C.snackZ = C.z;
        C.snackFallFromY = C.y - 0.5f - SNACK_H[stage().target] * 0.5f;
        C.dropSuccess = inChute(C.snackX, C.snackZ);   // outcome fixed now
        C.carrying = false;
```

```
                C.dropT = 0.0f;
                C.state = RESOLVING_DROP;
            }
        }
```

Listing 22.3. `requestGrabOrRelease()`.

Space does different things depending on ClawState, and any other state ignores it. The chute test is performed at the instant of release, and RESOLVING_DROP only animates the fall, so the recorded outcome cannot change afterwards.

Deterministic machine cues

```
static float pulseAt(float t, float t0, float dur) {
    if (t < t0 || t > t0 + dur) return 0.0f;
    return sinf((t - t0) / dur * PI_F);
}
...
case 0:   // brief display flicker + indicator blink
    c.dip = 0.8f * pulseAt(t, 0.4f, 0.15f);
    c.indicator = pulseAt(t, 0.4f, 0.20f);
    break;
```

Listing 22.4. Cue pulses in `computeCues()`.

computeCues() is a pure function of the scene clock K.cueT and the stage number. It has no random numbers and no side effects. The renderer reads the returned intensities, so a cue can never change the credit, the target, the attempts or the player's input.

Ordered outro phases

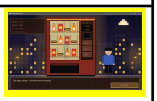
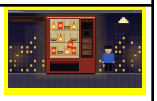
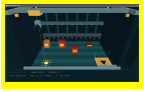
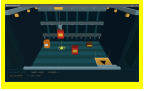
```
static void updateOutro(float dt) {
    K.stateT += dt;
    while (G.outroPhase <= OUT_FLICKER && K.stateT >= OUTRO_DUR[G.outroPhase]) {
        K.stateT -= OUTRO_DUR[G.outroPhase];          // boundaries crossed in order, once
        G.outroPhase++;
    }
    if (G.outroPhase > OUT_FLICKER) returnToTitle();  // ordinary PLEASE INSERT COIN screen
}
```

Listing 22.5. `updateOutro()`.

The while loop handles a frame long enough to cross more than one phase boundary by consuming the durations in order, so each phase starts exactly once. Only this update function advances the sequence; drawReflection() only reads the phase and the time.

Screenshots

The figures below are placeholders for screenshots captured from the running program. No screenshot has been inserted yet, and concept art must not be substituted. Capture each at the default 1280 x 720 window size from the compiled PLEASE_INSERT_COIN.exe, and replace the placeholder box with the image.

Figure	Game state	What the screenshot should show	File name	Status
23.1	TITLE	Night street, glowing machine, title panel, INSERT COIN and HOW TO PLAY	**[title.png]**	
23.2	CUSTOMER_INTRO	Customer beside the machine with the request in the dialogue band	**[intro.png]**	
23.3	MACHINE_2D	Purchase interface: coin inserted (CREDIT: 1/1) and the snack buttons	**[purchase.png]**	
23.4	TRANSITION_TO_3D	The jam with DISPENSING INTERRUPTED; optionally one frame of the warning border and one of the camera shift	**[transition.png]**	
23.5	RESCUE_3D	Interior view with claw, marker, target outline, chute and maintenance terminal	**[rescue.png]**	
23.6	RESCUE_3D carrying	Claw holding the snack above the floor (SPACE: RELEASE hint visible)	**[carry.png]**	
23.7	RESCUE_3D stage 3	The moving restocking arm between the claw and the juice carton	**[arm.png]**	

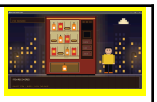
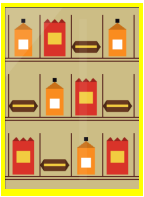
23.8	STAGE_RESULT	ITEM RECOVERED with the delivered snack in the chute, and the reaction line	**[success.png]**	
23.9	GAME_OVER	RECOVERY UNSUCCESSFUL with the Retry button	**[failure.png]**	
23.10	Pause overlay	Darkened frozen rescue with the four menu options	**[pause.png]**	
23.11	REFLECTION_OUTRO	Reflection in the glass during the turn (and, if possible, the flicker)	**[reflection.png]**	
23.12	Vertex arrays	A view where shelf rails (2D) and the cuboid scene (3D) are visible, to accompany Section 20	**[arrays.png]**	 

Table 23.1. Screenshot checklist.



Figure 23.1. Title screen.

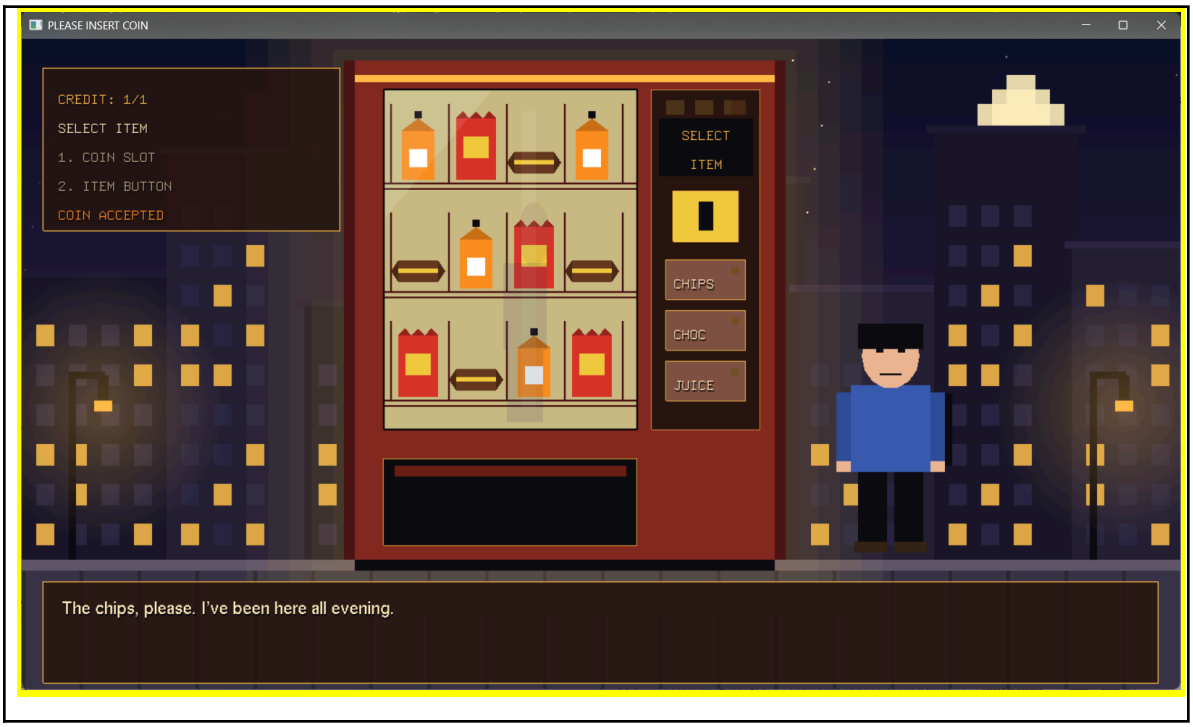


Figure 23.3. Purchase interface.

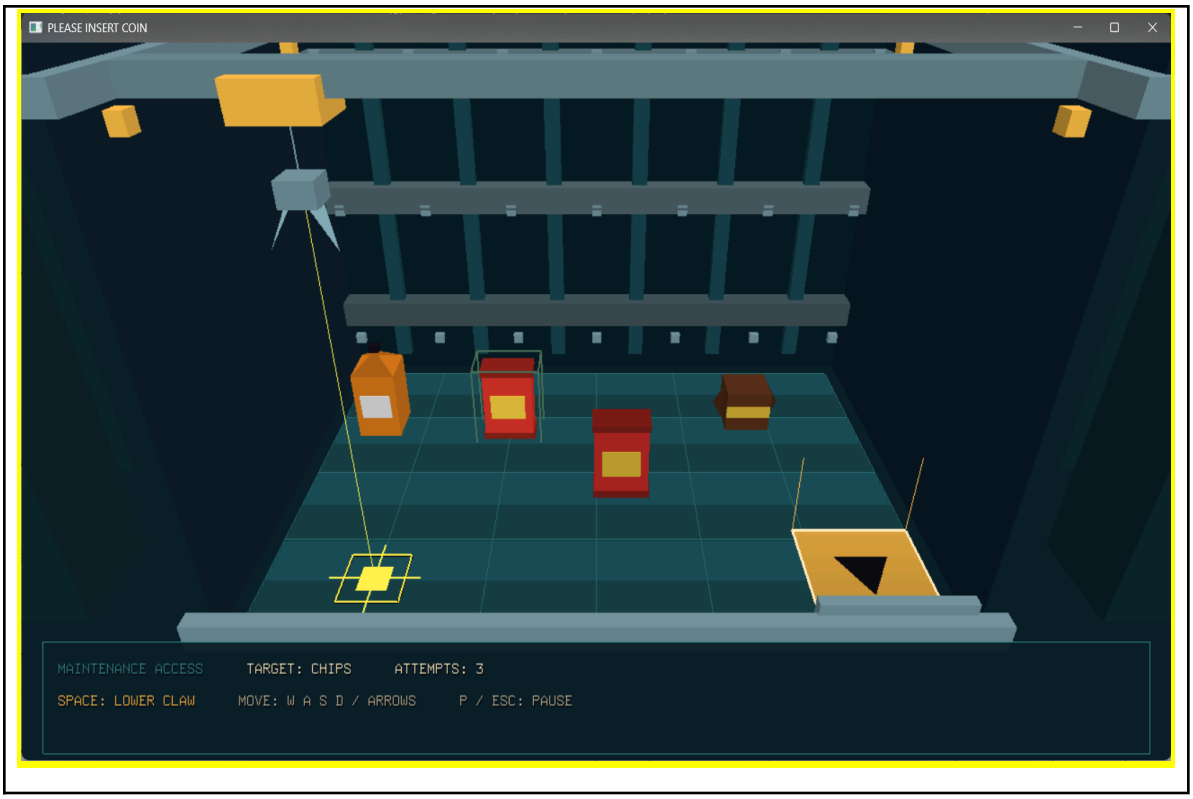
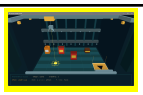


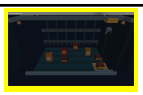
Figure 23.5. 3D rescue view.

Placeholder boxes are shown for three figures as examples; add the remaining figures in the same way, using the captions in Table 23.1.

10+ Test Cases

The cases below are taken from the development test matrix in the project handoff and adapted to the implemented controls. **None of them has been executed and recorded for this report.** The Actual Result column is intentionally empty and every status is UNVERIFIED. A status may be changed to PASS or FAIL only after I run the case on the compiled program and record what I observe, with a screenshot or note as evidence.

ID	Scenario	Action / input	Expected result	Actual result	Status
T01	Launch	Run the executable and look at the title screen	Window opens; title text, machine, street and buttons are visible; at least five objects and three primitive types can be identified		VERIFIED
T03	Normal purchase	Start, continue, click the coin slot, click the requested snack	One transition to the 3D rescue begins and the rescue starts only after it		VERIFIED
T04	Selection without a coin	Click any snack before inserting a coin	No transition; INSERT COIN FIRST feedback; attempts unchanged		VERIFIED
T05	Duplicate coin / wrong snack	Click the coin slot twice, click a wrong snack, then the correct one	Only one credit; the wrong snack keeps the credit; the correct snack proceeds once		VERIFIED
T06	Movement	Hold and release W/A/S/D and arrows; press opposite keys; press two keys for a diagonal	Correct X/Z directions; immediate stop; opposite keys cancel; diagonal speed not faster		VERIFIED
T07	Boundaries and solids	Push the claw into each wall and package, empty and carrying	The claw/snack does not leave the compartment or pass through packages or the arm		VERIFIED

T08	Grab tolerance	Grab with the claw centred, near the tolerance edge, and clearly off the target	Target taken inside tolerance; a miss is charged once; no other package can be attached		VERIFIED
T09	Space held during a miss	Hold Space during a missed grab	Exactly one attempt is spent; a new cycle needs a new press		VERIFIED
T11	Chute delivery	Carry the snack over the chute and release	ITEM RECOVERED follows; no attempt is spent		VERIFIED
T12	Invalid drop	Release just outside the chute	One attempt is spent; claw and target reset; no delayed success		VERIFIED
T13	Pause and resume	Pause during a descent, wait, then resume	Everything freezes; the same progress resumes without a jump; movement needs a fresh key press		VERIFIED
T14	Exhaustion and retry	Fail until zero attempts; press R, then Enter, then click Retry (separate runs)	RECOVERY UNSUCCESSFUL; each input restores the same stage with full attempts		VERIFIED
T15	Collection	After delivery, click or press Enter repeatedly	One input collects; the next, separate input continues; the stage advances once		VERIFIED
T16	Invalid input for the state	Click purchase buttons during rescue; press R during rescue; move keys on the title screen	Nothing happens; attempts, credit and stage are unchanged		VERIFIED
T17	Three stages	Complete stages 1 to 3	Correct customer, target and layout in order; fresh attempts each stage		VERIFIED
T19	Ending	Finish stage 3 and watch the ending	Customer leaves; reflection lingers, turns and vanishes with a flicker; title returns with no explanation		VERIFIED

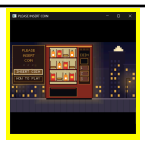
T36	Confirmed restart	Open the pause menu, choose REINITIALIZE, cancel, then repeat and confirm	Cancel changes nothing; confirm restores the stage attempts and layout		VERIFIED
T02	Resize and mouse	Resize the window and click the coin slot and snack buttons	Hit areas match the drawn controls; clicks in the black bars do nothing		VERIFIED

Table 24.1. Proposed test cases (18 cases; all unexecuted).

Errors/Problems Encountered

The problems below are the ones that I can document from the development record. They are limited to what actually appears in the development session; I will add the problems I encountered myself in the last subsection.

Compilation and build

- **Unused function warning.** Compiling with `-Wall -Wextra` reported that `drawScene2DStatic()` was defined but not used.
- **Runtime library.** The program links to FreeGLUT dynamically, because no static FreeGLUT library was installed in the MSYS2 UCRT64 setup, so the executable needs `libfreeglut.dll` beside it. An attempt to link statically failed with "cannot find -lfreeglut_static".

Rendering and layout issues found while inspecting the first running build

- The first 3D camera was too high and far, which made the floor small and left the top gantry cluttered.
- The rescue heads-up panel at the top-left covered the claw when it was parked near the front.
- The stage-3 arm's support column rose almost to the ceiling and hid part of the scene.
- The final reflection was too faint on the cream-coloured glass to be noticed.

Defects reported in a code review of the first complete version

A review of the first complete source reported the following defects.

#	Defect
1	Collision footprints did not match the drawn packages: all packages used one square size, so the carried chocolate's wrapper could extend beyond the wall, and chips blocked more than their visible width.
2	Text escaped its panels when the window was made smaller (for example the wrong-selection feedback, the restart label and the help text).
3	Moving the mouse pointer out of the window cleared the held movement keys, because the pointer-entry callback was treated as a focus change.
4	Movement keys pressed during a descent were stored and then moved the claw after it finished ascending without a new key press.
5	On resume, the time baseline was not refreshed, so delayed timer delivery could apply paused time as movement.
6	Collecting an item restarted the black fade of the result screen.
7	The target highlight and chute pulse kept animating while the rescue was paused.
8	After an attempt was resolved inside the fixed-step loop, the time accumulator could become negative.
9	The jam animation removed the wrong shelf item for stages 1 and 3, so the snack appeared twice.
10	The game-over backdrop showed the reset start layout instead of the failed pose.

Table 25.1. Review findings.

Problems from my own development experience

I had problems on how to properly render the 3D plane and make it visually appealing, I also considered more narrative aspects within the gameplay which I wasn't able to implement since I lack the skill on how to properly shape them in OpenGL, there also wasn't enough time to learn to do all that I wanted to do.

How You Solved the Problems

Problem	Solution	Checked how
Unused function (25.1)	Removed drawScene2DStatic().	Compiled with no warnings.

Runtime DLL (25.1)	Kept the dynamic link and shipped libfreeglut.dll beside the executable.	DLL dependency confirmed with objdump; the executable was not launched without the DLL.
Camera, HUD, arm column, faint reflection (25.2)	Lowered and tightened the camera (begin3D()), moved the rescue terminal to a bottom band, shortened the arm column, and darkened the reflection tint and raised its alpha.	Inspected in screenshots; compiled.
1 Collision footprints	Added SNACK_HX/SNACK_HZ per package type and rectangular footprintBlocked(); the carried footprint combines claw and snack extents.	Compiled. Routes were checked by hand against the layouts; not played through.
2 Text overflow	Added wrapText(), drawWrapped(), textBlock(), short labels and fit checks.	Inspected at 640 x 360 and 320 x 180; compiled.
3 Pointer leaving the window	Removed the entry callback; keys are cleared on hidden window or loss of foreground.	Compiled; not tested by hand.
4 Buffered movement	movementAccepted() gates held-key recording, and starting a grab or release calls clearHeldKeys().	Compiled; not tested by hand.
5 Resume baseline	resumeRescue() refreshes gLastMs.	Compiled; the delayed-timer case was not reproduced.
6 Fade on collect	Removed the clock reset in acknowledgeResult().	Compiled; not tested by hand.
7 Paused animation	Added the visual clock K.visT, which is frozen while the rescue is paused or failed.	Compiled; not tested by hand.
8 Negative accumulator	The fixed step is subtracted before stepRescue() runs.	Compiled; not tested by hand.
9 Jam duplicate	jamSlot() chooses the shelf slot from the current target for both removal and jam placement.	Compiled; not re-inspected after the change.
10 Game-over backdrop	resolveFailure() enters GAME_OVER before resetting when attempts reach zero.	Inspected at 640 x 360; compiled.

Table 26.1. Problems, solutions and the extent of checking.

Items marked "not tested by hand" should be confirmed with the test cases in Section 24 before the report states that they are resolved.

How ChatGPT Was Used

The project handoff states that it was prepared from earlier planning conversations and that the submission requires the complete ChatGPT conversation as a deliverable. Based on that record and the saved prompt in my course folder, I used ChatGPT in the following ways:

- **Brainstorming.** My opening prompt (saved as midterm_viray.md) asked for help brainstorming a project and explicitly said that I would choose the final idea myself. The vending-machine concept was my selection.
- **Requirement interpretation.** The handoff lists the assignment requirements as R01-R12 and separates official requirements from design decisions and working defaults.
- **Technical planning.** The handoff contains a six-effort build plan (M0-M6), a proposed state machine, input rules, the vertex-array plan, and a test matrix of 51 proposed cases (T01-T51).
- **Visual design and writing.** The handoff records the retro 2D and low-poly 3D visual direction, the diegetic interface, and the editable customer dialogue.
- **Documentation.** Early drafts of Sections 1-10 of this report were drafted with ChatGPT and then manually revised by me.

Implementation and review. The planning conversation in ChatGPT did not write the final C++ program. The source PLEASE_INSERT_COIN.cpp was written and revised in a separate coding session in Claude Code (Anthropic), working from the handoff. That session also produced the revisions described in Section 26 and drafted this Sections 11-31 text.

AI Suggestions Accepted

The handoff marks which decisions I confirmed. The following were confirmed by me and are present in the implemented game:

Suggestion	Evidence of acceptance	Where it is in the program

Mini-game with a routine 2D purchase that switches to a 3D rescue	Marked "confirmed by the user" in the handoff	TRANSITION_TO_3D, begin3D()
Maintenance claw with precise, forgiving, non-drifting control	Confirmed in the handoff	moveClaw(), GRAB_TOL_X/Z
Limited attempts with no countdown	Confirmed in the handoff	resolveFailure(), START_ATTEMPTS
Three customers with increasingly odd requests; mystery told only visually	Confirmed in the handoff	STAGES[], computeCues()
Final reflection that lingers, turns and vanishes in a flicker	Confirmed in the handoff	REFLECTION_OUTRO, drawReflection()
Retro pixel-style 2D street and low-poly 3D interior; fixed elevated oblique camera with a claw marker	Confirmed visual direction in the handoff	drawStreet2D(), drawScene3D()
Explicit state machine rather than separate boolean flags	Part of the approved structure in the handoff	GameState, ClawState, Overlay
Vertex arrays for repeated geometry, with both draw paths	Adopted as the project recommendation in the handoff	drawRails2D(), drawGrid3D(), drawCuboid()
Explicit item collection before the next customer	Retained as the selected default in the handoff	acknowledgeResult()

Table 28.1. Accepted suggestions supported by the handoff.

AI Suggestions Rejected/Modified

Only changes that the handoff or the project record documents are listed. I have not added disagreements that I cannot support.

Suggestion	What changed	Evidence
"CRITICAL SNACK FAILURE" as the jam message	Replaced by the more restrained DISPENSING INTERRUPTED	Handoff Section 8 notes the earlier example as superseded
Three attempts per stage	Kept only as an adjustable default (START_ATTEMPTS); only "limited attempts" was confirmed	Handoff Section 3

Framebuffer pixelation at 320 x 180	Not implemented in the delivered version; direct rendering was used	Handoff marks it as a later enhancement and the source has no framebuffer code
Customer dialogue examples	Treated as editable text, not a locked script	Handoff Section 5

Table 29.1. Modified or deferred suggestions.

What You Learned

OpenGL rendering

Project evidence: The project required building every object from primitives (Section 11), setting up 2D orthographic and 3D perspective passes (begin2D(), begin3D()), and restoring state between passes.

Building PLEASE INSERT COIN taught me that everything on screen comes from vertices submitted as primitives. The street, the machine, the customers and the 3D interior are all made from points, lines, triangles, triangle fans and quads, with no external models. I learned that each drawing pass needs its own setup. begin2D() uses an orthographic projection with depth testing off, while begin3D() uses gluPerspective(), gluLookAt() and depth testing. If a pass relied on state left over from the previous one, text and shapes could draw incorrectly, so each function sets what it needs. I also learned that a few small helpers such as fillRect(), fillTri() and fillEllipse() make a large scene manageable.

Coordinate systems

Project evidence: Three coordinate spaces had to be kept consistent: GLUT's top-left mouse pixels, the 320 x 180 bottom-left logical space, and the 3D world with X as width and Z as depth (including the mirrored-X correction).

This project made me deal with three coordinate systems at once. GLUT reports the mouse from the top-left in pixels. The 2D scene uses a 320 x 180 logical space with the origin at the bottom-left. The 3D world uses X for width, Y for height and Z for depth. I learned to convert between them deliberately. mouseToLogical() flips Y and maps through the letterboxed content rectangle. In 3D, looking along +Z mirrored the X axis, so I added

glScalef(-1, 1, 1) to make D and the right arrow move the claw to the right. I also learned to keep gameplay in world units, so that window size never changes movement speed, collision or the grab tolerance.

GLUT callbacks and event-driven programming

Project evidence: The game relies on display, reshape, keyboard, special-key, mouse, passive-motion and timer callbacks, with one timer chain driving all updates.

I learned that a GLUT program does not run in a loop I write. It reacts to callbacks: display, reshape, keyboard, special keys, mouse, passive motion and a timer. Key-down and key-up callbacks let me track held keys for smooth claw movement, while one-shot actions such as Space needed glutIgnoreKeyRepeat(1) so holding the key could not repeat them. I learned to keep callbacks thin: they check the state and call one named function such as insertCoin() or requestGrabOrRelease(). I also learned that display() should only draw, and that all time-based updates belong in one timer chain with a fixed time step.

Arrays and vertex arrays

Project evidence: Stage data, shelf rails and grid lines are stored in arrays, and the indexed cuboid mesh required deciding between 8 shared corners and 24 face-local records.

I learned that arrays organize the program in two ways. The STAGES[] array holds the three customers, targets and layouts, and loops draw repeated shelves, windows and packages. With vertex arrays, I learned the difference between glDrawArrays() and glDrawElements(). I used glDrawArrays() for shelf rails and the floor grid, and an indexed cuboid mesh for the 3D objects. I also learned why that mesh has 24 vertex records instead of 8 corners. Each face has its own shade colour, so a shared corner has to be stored once per face. The indices then let each face reuse its records for two triangles, and one mesh serves every box in the scene.

Debugging

Project evidence: The review findings in Section 25 involved timing, input state and layout problems that were not visible from static code alone.

I learned that many problems are not visible in the code until the program runs, or until someone reviews it. A code review of my first version found issues with timing, input state and layout. For example, movement pressed during a descent was stored and then moved the claw later, and text spilled out of panels when the window got smaller. Fixing them taught me to ask what state a value is left in after a pause, a reset or a state change. It also taught me to test at different window sizes. I also learned to be honest about what I have and haven't tested. Some fixes were only compiled, so I recorded them that way in Section 26.

Software design

Project evidence: Separating state ownership, input adapters and read-only renderers, and routing every failure through one function, made the rules easier to keep consistent.

I learned to design the program around state. A single GameState value, a separate ClawState and an overlay for menus made it clear which input is allowed at any moment. Routing every failed attempt through resolveFailure(), with a guard so one cycle costs only one attempt, kept the attempt rules consistent. Keeping the renderers read-only and the cue timings as pure functions of the clock meant that visual effects could not change gameplay. Keeping everything in one file with commented sections also made it easier to navigate and explain.

Conclusion

PLEASE INSERT COIN was designed as a short interactive mini-game in which a routine 2D vending-machine purchase becomes a 3D claw rescue. The objective was to apply C++, OpenGL and FreeGLUT concepts from Modules 01 to 04 in a small system that responds to the user, rather than a static drawing.

The implemented program is a single C++ source file with an explicit state machine of nine game states and an overlay for pause, help and confirmations. It builds its world from seven OpenGL primitive types, uses bitmap text, keyboard and mouse callbacks, and a single timer chain, and demonstrates both glDrawArrays() and an indexed cuboid mesh drawn with glDrawElements(). Beyond the basic requirements, it adds the 2D-to-3D transition,

a claw with collision detection and limited attempts, three stages including a moving obstacle, and a short visual mystery that ends on a reflection in the machine glass.

Some goals in the handoff were not implemented, notably uniform low-resolution pixelation through a framebuffer and optional fog. The program compiles without warnings and was inspected in a scripted run that captured screenshots of the main states.